TWO SUM PROBLEM

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array nums[] containing only positive integers, followed by an integer target. The array must be passed to a function named two_sum() using its address, with pointers used to pass the array between the calling function and two_sum().

The function two_sum() must find two distinct indices i and j such that nums[i] + nums[j] = target, and display them separated by a space. The indices may be returned in any order. If no such pair exists, the function must display -1 -1. A nested-loop approach must be used, and at most one valid solution is assumed to exist.

2. Aim

To develop a C program that uses pointers, dynamic memory allocation and nested loops to find the indices of two numbers in an array whose sum equals a given target value.

3. Concepts Used

- Arrays and dynamic memory allocation using malloc() and free().
- Pointers and pointer arithmetic (*(nums + i)) to access array elements.
- Passing an array to a function by its address.
- Functions with a void return type.
- Nested for loops and conditional (if) statements.

4. Algorithm

1. Start the program.
2. Read the size of the array and store it in size.
3. Allocate memory for size integers using malloc().
4. Read the array elements and the target value.
5. Call two_sum(nums, size, target), passing the base address of the array.
6. In two_sum(), run an outer loop with i from 0 to size - 2.
7. For each i, run an inner loop with j from i + 1 to size - 1.
8. If *(nums + i) + *(nums + j) equals target, print i and j, then return.
9. If both loops finish without a match, print -1 -1.
10. Free the allocated memory and stop the program.

5. Source Code

File name: two_sum.c

```
#include <stdio.h>
#include <stdlib.h>

/* Function prototype */
void two_sum(int *nums, int size, int target);

int main(void)
{
    int size, target, i;
    int *nums;

    /* Read the size of the array */
    printf("Enter the size of the array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid size.\n");
        return 1;
    }

    /* Allocate memory for a variable-length array */
    nums = (int *)malloc(size * sizeof(int));
    if (nums == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    /* Read the array elements using pointers */
    printf("Enter %d positive integers: ", size);
    for (i = 0; i < size; i++)
    {
        scanf("%d", nums + i);
    }

    /* Read the target value */
    printf("Enter the target: ");
    scanf("%d", &target);

    /* Pass the address of the array to two_sum() */
    printf("Output:\n");
    two_sum(nums, size, target);

    free(nums);
    return 0;
}
```

```c
/* Finds two distinct indices whose elements add up to target */
void two_sum(int *nums, int size, int target)
{
    int i, j;

    for (i = 0; i < size - 1; i++)
    {
        for (j = i + 1; j < size; j++)
        {
            if (*(nums + i) + *(nums + j) == target)
            {
                printf("%d %d\n", i, j);
                return;
            }
        }
    }

    /* No valid pair found */
    printf("-1 -1\n");
}
```

6. Explanation of the Program

main() function

The program first reads the size of the array and checks that it is greater than zero. Memory for the array is then allocated dynamically so that the array can have any length chosen by the user. The elements are read using scanf("%d", nums + i), where nums + i is the address of the i-th element. After reading the target, main() calls two_sum() with the pointer nums, which holds the base address of the array.

two_sum() function

The function receives the array as an int pointer. The outer loop picks each element at index i, and the inner loop starts from i + 1 so that the same element is never used twice and no pair is checked more than once. The elements are accessed using pointer notation *(nums + i) and *(nums + j). When a pair with the required sum is found, its indices are printed and the function returns immediately. If no pair is found, -1 -1 is printed.

Time and space complexity

Because of the nested loops, the time complexity is $O(n^2)$ in the worst case. Apart from the input array, only a few integer variables are used, so the extra space required is O(1).

7. Dry Run (Example 1)

Array: 2 7 11 15, Target: 9

i	j	nums[i]	nums[j]	Sum = 9 ?
0	1	2	7	2 + 7 = 9, Yes

At i = 0 and j = 1, nums[0] + nums[1] = 2 + 7 = 9, which equals the target. The program prints 0 1 and returns.

8. Sample Input and Output

The program was compiled with gcc -Wall -Wextra two_sum.c -o two_sum and executed for the four given examples. The output of each run is shown below.

Test Case 1

```
Enter the size of the array: 4
Enter 4 positive integers: 2 7 11 15
Enter the target: 9
Output:
0 1
```

Test Case 2

```
Enter the size of the array: 3
Enter 3 positive integers: 3 2 4
Enter the target: 6
Output:
1 2
```

Test Case 3

```
Enter the size of the array: 2
Enter 2 positive integers: 3 3
Enter the target: 6
Output:
0 1
```

Test Case 4

```
Enter the size of the array: 3
Enter 3 positive integers: 1 2 3
Enter the target: 7
Output:
-1 -1
```

9. Summary of Test Cases

Test	Size	Array	Target	Output	Result
1	4	2 7 11 15	9	0 1	Passed
2	3	3 2 4	6	1 2	Passed
3	2	3 3	6	0 1	Passed
4	3	1 2 3	7	-1 -1	Passed

10. Conclusion

The program successfully reads a variable-length array, passes it to two_sum() through a pointer, and uses a nested-loop approach to find two distinct indices whose elements add up to the target. It prints the indices when a pair exists and prints -1 -1 otherwise. The output matched the expected result in all four test cases. This project helped in understanding pointers, dynamic memory allocation, passing arrays to functions and nested loops in C.